Verifiable Containment Attestation After LLM Guardrail Failure

Anonymous Author(s)

ABSTRACT

LLM guardrails are commonly evaluated at the point where a user input is accepted or blocked. Agentic failures, however, can occur after this first gate has opened: untrusted retrieved content may gain authority, model output may request a tool, or unsafe text may be exported. This paper presents a provider-agnostic post-guardrail containment and evidence system. The prototype enforces input, context-provenance, tool-invocation, and output-leakage boundaries, then emits metadata-only evidence bundles signed with Ed25519 over canonical JSON. Our motivating observation uses a real input-only guardrail: a frozen live Llama Guard 4 12B capture allowed 138 of 150 malicious cases on a 180-case corpus, not because it is weak but because the malicious instruction is structurally located downstream of the user turn it inspects. The gateway then contained all 138 downstream (contained fraction 138/138, 95% Clopper-Pearson CI [0.97, 1.0]). On the same corpus the gateway blocked 30/30 direct-input attacks, passed 30 adversarially-shaped benign hard negatives with no false blocks (0/30, 95% CI [0, 0.12]), and recorded no unauthorized tool execution, context-authority escalation, or unsafe export. On an independently-authored attack corpus rendered from the AgentDojo benchmark (175 payloads we did not write), the deterministic input firewall detects only 35/175 (95% CI [0.14, 0.27]) yet all 175 are structurally contained downstream with no untrusted context gaining authority—the same thesis under third-party attacks. A signed VCA bundle verifies offline and rejects tampering, and a synthetic recorded-signal fixture confirms that downstream containment is invariant to the external advisory verdict. We are deliberate about scope: the corpus is synthetic and self-authored, the single real external guardrail is one model, and the signature proves issuer and integrity, not producer honesty, jailbreak immunity, production readiness, model safety, or vendor ranking.

CCS CONCEPTS

• **Security and privacy** → **Systems security**; • **Computing methodologies** → *Artificial intelligence.*

KEYWORDS

LLM security, prompt injection, agentic systems, guardrails, attestation, reproducibility

ACM Reference Format:

Anonymous Author(s). 2026. Verifiable Containment Attestation After LLM Guardrail Failure. In *19th ACM Workshop on Artificial Intelligence and Security (AISec '26).* ACM, New York, NY, USA, 6 pages.

AISec '26, October 2026, TBD

1 INTRODUCTION

Input guardrails are necessary, but they are not a complete security boundary for agentic LLM systems. A direct jailbreak may be visible in the user input. An indirect prompt injection may instead arrive through retrieved data, tool output, or model-generated text that is later interpreted as an action request. Prior work shows that LLM-integrated applications can treat untrusted data as instructions [2]; benchmark environments such as AgentDojo show that tool-using agents require explicit security evaluation under untrusted context [1].

This paper studies a narrower systems question: after a guardrail misses, can a gateway produce machine-checkable evidence of what did and did not happen downstream? We do not propose a better jailbreak detector. We ask whether downstream containment can be made explicit, replayable, and cryptographically verifiable. The intended reader is a reviewer, operator, or auditor who should not have to trust a live service to know whether untrusted context gained authority, an unauthorized tool executed, or unsafe output was exported.

The VCA prototype enforces four boundaries: an input firewall, a context-provenance guard, a tool-invocation gate, and an output-leakage firewall. It treats model output and external guardrail verdicts as advisory. Each run emits metadata-only receipts and audit events. A second layer packages run-set evidence as canonical JSON and signs it with Ed25519 [3, 6], using public verification keys and deterministic reproduction checks.

We make three contributions:

(1) A post-guardrail containment gateway that makes downstream consequences—context-authority escalation, tool self-authorization, and unsafe export—the unit of evaluation, with boundary decisions taken by deterministic code rather than the model under inspection.
(2) A verifiable containment attestation format that turns those boundary outcomes into metadata-only canonical evidence bundles signed with Ed25519, with public verification, offline reproduction, tamper rejection, and machine-readable non-claims, under an explicitly stated honest-producer trust boundary.
(3) An evidence-first evaluation, including a real input-only guardrail capture that quantifies the downstream blind spot, a synthetic advisory-invariance check, and a public reproduction chain, paired with a claim-governance pattern that prevents bounded evidence from being read as a broad safety claim.

2 BACKGROUND AND MOTIVATION

Prompt injection is a first-order LLM application risk in practitioner guidance such as OWASP's LLM Top 10 [5]. The key distinction for this work is not whether prompt injection exists, but where it enters an agentic workflow. Direct injection targets the user turn. Indirect injection targets content later retrieved or processed by the application. Tool-using agents add a further boundary because model output can be interpreted as an action request unless a separate policy gate checks authorization.

Most guardrail evaluations report whether an input is blocked, warned, or allowed. That is useful, but it does not answer the downstream consequence question. If an input-only guardrail allows a benign-looking task while the malicious instruction lives in untrusted context, then the relevant security property is whether that context can gain authority. If a model asks to execute a tool, the relevant property is whether the tool request is authorized outside the model. If model output contains unsafe material, the relevant property is whether it reaches the export boundary.

The VCA prototype therefore treats containment evidence as the primary artifact. It records boundary outcomes without publishing raw harmful transcripts or raw model outputs. This evidence-first framing is also a reproducibility requirement: a reviewer should be able to verify the artifact offline, with public keys and scripts, rather than trusting a live provider or private deployment.

3 RELATED WORK

Prompt injection in LLM applications. Indirect prompt injection showed that LLM-integrated applications blur the boundary between data and instructions: content retrieved from email, webpages, documents, or plugins can become operational instruction text once it reaches the model [2]. USENIX Security work has since formalized prompt-injection attacks and defenses, showing that case studies alone are insufficient and that defenses should be evaluated against explicit tasks, attack classes, and security properties [4]. Practitioner taxonomies, including OWASP LLM01, similarly separate direct and indirect injection and emphasize least privilege, output validation, human approval for high-risk actions, and segregation of untrusted content [5]. Our work follows this systems view but evaluates a different question: after the input guardrail misses, can downstream containment and its evidence be verified independently?

Agent benchmarks and tool-use security. AgentDojo evaluates LLM agents that operate tools over untrusted data and provides an extensible benchmark with realistic tasks, attacks, and defenses [1]. This line of work is essential for measuring whether agents complete tasks while resisting prompt injection. The VCA prototype is complementary: it does not claim a broad benchmark or a better attack detector. Instead, it records whether specific downstream consequences occurred, including context authority escalation, unauthorized tool execution, and unsafe output export, and then signs the evidence for offline verification.

Guardrails and audit evidence. Guardrails are usually evaluated as online decisions: input accepted, warned, or blocked. That frame is useful but incomplete for agentic systems because security-relevant outcomes can happen after acceptance. Our design borrows from security audit practice: preserve metadata, bind evidence with hashes and signatures, verify offline, and state non-claims. The use of canonical JSON and Ed25519 follows established standards for deterministic signing and verification [3, 6]. The novelty is not the cryptography itself; it is applying signed, machine-readable containment evidence to post-guardrail LLM security claims.

4 THREAT MODEL AND NON-CLAIMS

The defender operates an agentic LLM gateway between user input, retrieved context, model output, tool execution, and final export. The assets are tool authority, confidential context, system/developer instructions, unsafe output surfaces, and the integrity of exported evidence. The adversary can submit direct prompt-injection text, place malicious instructions in untrusted context, cause unauthorized tool requests, attempt output leakage, and tamper with exported evidence after publication.

Out of scope are kernel compromise, malicious system operators, private-key theft, production deployment assurance, and formal proof of semantic safety. Signed evidence proves that an exported bundle was issued by the signing-key holder and was not modified after signing; it does not prove that the live service was uncompromised, that the signing key was never stolen, or that all possible attacks were covered. Table 1 records the main paper claims and their boundaries.

5 SYSTEM DESIGN

Figure 1 shows the gateway. The input firewall handles direct attacks. If input passes, the context-provenance guard classifies each context item by origin and trust level; untrusted context can be passed as data but cannot acquire system or developer authority. Provider output is treated as untrusted material. Tool requests must pass the tool gate; final output must pass the output firewall before export. Table 2 summarizes each boundary, the attack class it addresses, its enforcement action, and the evidence it emits.

The design intentionally avoids using the model as its own security monitor. Boundary decisions are made by deterministic gateway code over structured metadata. The model and any external guardrail can influence the ordinary application response, but neither can authorize a tool, relabel untrusted context as trusted instruction, or suppress an audit event. This separation is what makes the later attestation meaningful: the evidence records gateway decisions rather than model assertions about those decisions.

Figure 2 shows the context-provenance authority decision in more detail. A context item carries an origin and a declared trust level; an invalid or unrecognized provenance is rejected, a trusted seed is accepted as context, and any untrusted origin (for example a tool result, an external reference, or provider output) is demoted to data so that it can inform the response but cannot acquire system or developer authority. Because this decision is made over structured metadata rather than free text, its outcome is recorded verbatim in the receipt, which is what a later verifier checks.

Evidence schema. Each boundary emits a compact record with a run identifier, boundary name, policy version, input/output digests where applicable, verdict, reason code, and non-claim tags. Raw prompts and raw provider outputs are excluded from the evidence packet. The VCA bundle aggregates these records with corpus metadata, metric summaries, privacy reports, referenced evidence hashes, public verification metadata, and a machine-readable list of limitations. This lets a reviewer check both positive claims, such as "no unauthorized tool execution in this corpus," and negative scope boundaries, such as "not a production deployment claim."

6 VERIFIABLE CONTAINMENT ATTESTATION

The attestation layer packages a run set as metadata-only evidence. A bundle contains metrics, boundary breakdowns, policy digests, privacy reports, referenced evidence hashes, and machine-readable

Table 1: Claims and non-claims. Evidence paths are repository-local and are verified in the paper package audit.

Claim	Evidence source	Non-claim boundary
Input-miss cases were downstream-contained on the frozen 180-case corpus	`stage-3l/metrics.json`	Does not imply general jailbreak resistance or production traffic validation.
The Stage 3M VCA bundle verifies offline and rejects tampering	`stage-3m/verifier-output.txt` and attestation files	Does not make signed evidence factual truth or prove the service was uncompromised.
The real external guardrail (Llama Guard 4) is one input-only reference; Stage 3V-A is a synthetic advisory-invariance fixture	`stage-3v-b/metrics.json`, `stage-3v/metrics.json`	One model, self-reported capture; the fixture carries no detection claim; not a vendor ranking.
The public VCA chain is replayable at tiered depths	`stage-3x/vca-chain-reproduction-results.json`	Not uniform 12/12 full reproduction.

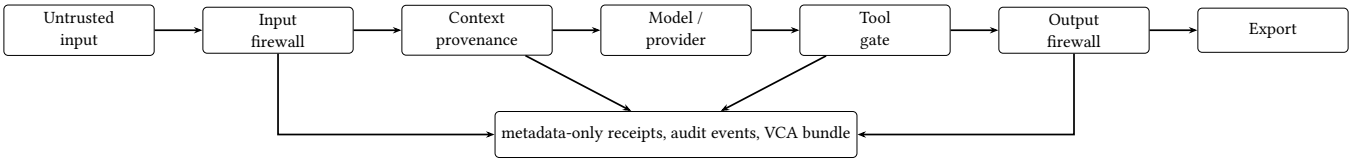

Figure 1: Post-guardrail containment pipeline. The defense acts after an input guardrail can fail. Provider output and external guardrail signals are advisory; final claims are derived from boundary evidence.

Table 2: Containment boundaries.

Boundary	Attack class	Enforcement action	Evidence emitted
Input firewall	Direct prompt injection	Block or warn before provider call	Input verdict, receipt, audit event
Context provenance	Indirect prompt injection	Demote untrusted context to data	Context verdict and boundary label
Tool gate	Tool self-authorization	Refuse unauthorized request	Tool verdict and no-execution evidence
Output firewall	Leakage/export pressure	Block unsafe export	Output verdict and output hash metadata

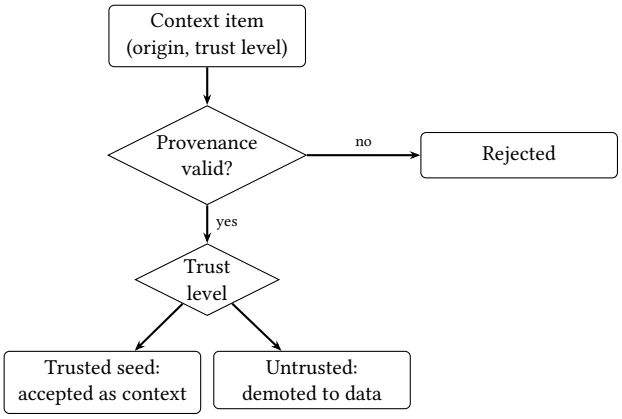

Figure 2: Context-provenance authority decision. Untrusted context may flow as data but never acquires system or developer authority, and neither provider output nor an external guardrail verdict can grant authority or relabel context. This deterministic decision—not the model—produces the boundary outcome that is later attested.

non-claims. The signature covers canonical JSON, allowing a verifier to check the signed content independently of whitespace or formatting changes. Verification has a portable mode for schema, signature, digest, public-key fingerprint, referenced file hashes, gate results, and leakage scans. Reproduction mode reruns deterministic producers where available.

7 EVALUATION

We use only repository evidence that can be traced to files and, where practical, commands. The evaluation ladder is shown in Figure 3; Table 3 summarizes the main results and Table 6 the reproducibility and tamper checks.

Stage 3L. A deterministic 180-case corpus is driven through real gateway boundary functions in pipeline order: prompt classification, context-provenance checks, tool gating, and output scanning. The corpus contains 120 input-miss downstream cases, 30 direct-input attacks, and 30 benign hard negatives. The main dependent variables are consequence metrics: whether untrusted context gained authority, whether an unauthorized tool executed, whether unsafe output was exported, whether receipts were emitted, and whether the audit chain verified. Because the corpus is synthetic and authored by us, the perfect malicious counts (120/120 input-miss contained, 30/30 direct blocked, 0/150 targeted attack success) are fixture-validity checks—they confirm the boundaries fire exactly as the case design intends—and not estimates of any real-world attack rate. To check that containment is not achieved by indiscriminate blocking, we report a benign false-positive rate over the 30 hard negatives, all adversarially shaped to resemble the malicious families

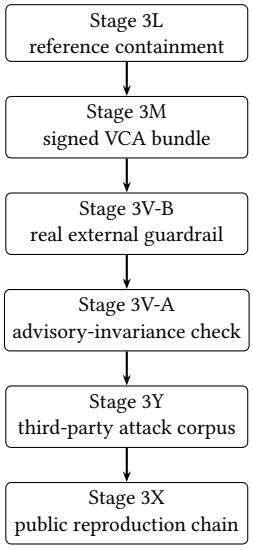

Figure 3: Evaluation ladder. Each stage contributes a different part of the paper claim and has a separate evidence artifact.

while remaining legitimate; none were blocked (0/30). We attach exact 95% Clopper-Pearson intervals throughout because the per-cell sample sizes are small: 0/30 bounds the benign false-positive rate at [0, 0.12] and 0/150 bounds targeted success at [0, 0.02], so these are upper-bounded near-zero results, not demonstrated true zeros.

Containment of the 120 input-miss cases is distributed across the downstream boundaries rather than concentrated at a single gate (Table 5): the context-provenance guard contained 72 cases, the tool gate 24, and the output firewall 24. The input firewall contained none of these by construction, because each input-miss case is designed to pass the input gate so that the downstream boundary under test is the one exercised.

Stage 3M. The Stage 3L run set is packaged into a metadata-only bundle signed with Ed25519. The public key and expected fingerprint are committed; the private key is not. The verifier checks structure, signature, evidence-file hashes, gate results, privacy, and reproduction matches. Tamper cases modify signed content, referenced metrics, and gate summaries to confirm that verification fails closed.

Stage 3V-B (the real external guardrail). A single real input-only guardrail anchors the external observation: a frozen live Llama Guard 4 12B capture (`meta-llama/Llama-Guard-4-12B`, one input-only greedy pass, temperature 0, 20 new tokens, 8-bit quantization, one NVIDIA RTX 3090 24GB GPU, transformers v4.51.3 Llama-Guard preview, PyTorch 2.8.0, CUDA 12.8). As an input-only signal it allowed 138 of 150 malicious cases. We read this not as a weakness of the model but as the structural blind spot of any input-only guardrail: in the 120 input-miss cases the malicious instruction is located downstream of the user turn the guardrail inspects. The gateway then contained all 138 downstream (138/138, 95% CI [0.97, 1.0]), and external-plus-gateway targeted success was 0/150. The frozen capture, not the model, is replayed in CI (`model_reexecuted_in_ci` is false); the live capture origin is self-reported and signed into evidence but not independently re-derived. This is one model and a boundary study, not a vendor comparison or ranking.

Stage 3V-A (advisory-invariance check, not a second guardrail). We have only one real external guardrail, so we do not claim a second comparator. Instead we use a synthetic, deterministic recorded-signal fixture—spanning allow, warn, and block verdicts that we authored—to test a structural property of the design: the gateway's downstream containment outcome does not depend on the external advisory verdict. Across the fixture the containment outcome was identical regardless of whether the synthetic signal said allow, warn, or block. This advisory-invariance is the property that lets the gateway treat any external guardrail, including a future stronger one, as untrusted advice rather than as authority. The fixture's verdicts are authored by us and carry no detection claim, so we report no attack-success rate for it.

Stage 3Y (independently-authored attack corpus). To break the self-authored circularity of Stage 3L, we render a third-party attack corpus from the AgentDojo benchmark [1]: 35 injection-task goals across four suites combined with five published attack envelopes (`ignore_previous`, `system_message`, `injecagent`, `important_instruct` `direct`), for 175 payloads authored by the AgentDojo team, not by us. We drive them through the real boundaries and separate content detection from structural containment (Table 4). The deterministic input firewall detects only 35/175 (95% CI [0.14, 0.27])—just the `injecagent` family, which contains an explicit override phrase—and misses 140/175 [0.73, 0.86]; even `ignore_previous` evades it because of a word insertion and a typo in AgentDojo's own string. The same 175 payloads, arriving as untrusted context, are structurally contained 175/175 [0.98, 1.0] with zero cases of untrusted context gaining authority, because provenance demotion is content-agnostic. This result is deliberately unflattering to the input layer and is the paper's thesis under independently-authored attacks: input guardrails miss, and downstream structural containment is what holds. Evidence is metadata-only; raw payloads are not committed.

Stage 3X. A public VCA timeline pins 12 signed rungs by tag and commit. We are explicit that "12/12" means twelve rungs verified at their declared tier, not twelve full re-executions: 3 rungs are re-run end-to-end, 7 are checked by evidence-hash, and 2 are index-only. The guarantee that holds across all three tiers, and across the historical evidence-hash formats, is the format-independent evidence-root digest.

8 SECURITY ANALYSIS

The system prevents a bounded class of downstream consequences in the evaluated corpora: untrusted context gaining authority, unauthorized tool execution, and unsafe output export. It does so by separating model text from policy authority. The evidence layer makes the resulting containment claim checkable offline.

The honest-producer trust boundary. The sharpest limit of any attestation is that it binds integrity and issuer, not truth. The gateway and its evidence producer sit inside the trusted computing base: a signature proves that the bundle was issued by the key holder and not modified afterward, but it cannot prove that the

Table 3: Main evaluation results.

Evaluation	Corpus / evidence	Key metric	Result	Limitation
Stage 3L	Frozen 180-case reference corpus	Input-miss downstream containment	120/120 (CI [0.97,1])	Synthetic, self-authored corpus; fixture-validity.
Stage 3L	Same	Direct-input attacks blocked	30/30	Does not imply general detector robustness.
Stage 3L	Same	Benign false-positive rate	0/30 (CI [0,0.12])	Hard-negative control; small-n upper bound.
Stage 3L	Same	Targeted attack success	0/150 (CI [0,0.02])	Self-authored corpus; not a world rate.
Stage 3L	Same	Unsafe tool/export/context escalation	0/0/0	Consequence metrics, not semantic safety proof.
Stage 3M	Signed VCA bundle	Offline verification	PASS	Signs evidence under honest-producer assumption; not factual truth.
Stage 3V-B	Real input-only guardrail (Llama Guard 4 12B)	Downstream-contained after guardrail miss	138/138 (CI [0.97,1])	One model; capture self-reported; not a ranking.
Stage 3V-A	Synthetic recorded-signal fixture	Advisory-invariance	invariant	Authored verdicts; no detection claim.
Stage 3X	Public reproduction packet	Rungs passed	12/12	Tiered replay (3 reproduce / 7 hash / 2 index).

Table 4: Stage 3Y: independently-authored AgentDojo attack corpus (175 payloads) through the real boundaries. Content detection is by the deterministic input firewall; structural containment is content-agnostic provenance demotion. Intervals are 95% Clopper-Pearson.

Boundary	Metric	Result (95% CI)	Reading
Input firewall	Content detection	35/175 [0.14, 0.27]	Only `injecagent` (explicit override phrase).
Input firewall	Missed	140/175 [0.73, 0.86]	Four of five envelopes evade the denylist.
Context provenance	Structural containment	175/175 [0.98, 1.0]	Untrusted demoted (170) or rejected (5).
Context provenance	Untrusted gained authority	0	Core invariant holds on third-party attacks.

Table 5: Per-boundary containment of the 120 Stage 3L input-miss cases. The input firewall contains none by construction; each case is built to pass the input gate and exercise one downstream boundary.

Downstream boundary	Input-miss cases contained
Context-provenance guard	72
Tool gate	24
Output firewall	24
Total downstream-contained	120

gateway emitted a faithful receipt for the run it actually performed. A gateway that executed an unsafe action and then signed a clean receipt would pass every check in this paper. We state this assumption explicitly rather than hide it: VCA is meaningful only insofar as the producer is honest and the signing key is uncompromised. Two design choices narrow, but do not close, this gap. First, boundary outcomes are computed by deterministic code over structured metadata, so the receipt records a mechanical decision rather than

Table 6: Reproducibility and tamper checks.

Check	Expected	Verified	Command / source
Stage 3M verifier	PASS	PASS	`stage-3m/verifier-output.txt`
Evidence leakage	zero	true	Stage 3M verifier output
Bundle digest	match	true	Stage 3M verifier output
Stage 3X timeline	PASS	true	`vca-chain-reproduction-results.json`
Tier summary	3 reproduce, 7 hash, 2 index	matched	Stage 3X results JSON

a model self-report. Second, the bundle binds referenced evidence by hash and the verifier recomputes those hashes, so post-hoc edits are rejected. Closing the gap requires an independent oracle that observes consequences without trusting the producer; we regard producer-independent witnessing as the central direction for future work.

The system does not prove semantic harmlessness. An attacker outside the threat model could compromise the signing key, the host, or the evidence producer. A malicious operator could choose not to record evidence. A broader real-world deployment would also need operational key management, monitoring, and incident response. These are deployment concerns outside the paper's research prototype claim.

9 ETHICS AND RESPONSIBLE RELEASE

This is defensive research. The paper and artifact avoid releasing raw harmful jailbreak transcripts or operational payloads. The corpora are synthetic or redacted. The evidence is metadata-only and omits raw prompts, raw provider outputs, private keys, and real user data. The external guardrail study is vendor-neutral and must not be read as a ranking. The prototype is not deployment-ready or compliance-approved. These boundaries are stated in the paper, source notes, and artifact README.

10 LIMITATIONS AND FUTURE WORK

The main external-validity limitation is that the evaluation uses frozen synthetic corpora and replay artifacts, not production traffic; the perfect per-cell counts are fixture-validity checks on a self-authored corpus, reported with 95% Clopper-Pearson intervals that remain upper-bounded near-zero results on small samples rather than demonstrated true zeros. The external observation rests on a single real guardrail (one model); the second external artifact is a synthetic advisory-invariance fixture, not an independent guardrail, so we make no multi-guardrail or comparative claim. The live model capture origin is self-reported and signed into evidence, but the verifier does not rehash model weights or prove the original GPU environment. Stage 3X reports uneven replay depth; this is honest, but it means the public chain is not uniformly fully reproduced. Most fundamentally, as the security analysis states, the attestation assumes an honest evidence producer: a compromised or dishonest gateway is outside what a signature can detect. Future work should therefore prioritize producer-independent witnessing, alongside third-party artifact evaluation, broader and non-synthetic benchmarks, additional real external guardrails, deployment key-management analysis, and independent replication on further agent stacks.

11 CONCLUSION

Input guardrails will sometimes miss. The VCA prototype evaluates and attests what happened downstream after the miss: whether untrusted context gained authority, whether unauthorized tools executed, whether unsafe output was exported, and whether the evidence verifies offline. The contribution is not jailbreak immunity. It is a bounded systems pattern for converting post-guardrail containment from a verbal assurance into a machine-checkable artifact.

REFERENCES

[1] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. NeurIPS 2024 Datasets and Benchmarks Track. https://openreview.net/forum?id=m1YYAQjO3w

[2] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*. Association for Computing Machinery, New York, NY, USA, 79–90. https://doi.org/10.1145/3605764.3623985

[3] Simon Josefsson and Ilari Liusvaara. 2017. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032. https://www.rfc-editor.org/rfc/rfc8032

[4] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, USA, 1831–1847. https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[5] OWASP Gen AI Security Project. 2025. OWASP Top 10 for Large Language Model Applications 2025. https://genai.owasp.org/llm-top-10/. Accessed 2026-06-24.

[6] Anders Rundgren, Bret Jordan, and Samuel Erdtman. 2020. JSON Canonicalization Scheme (JCS). RFC 8785. https://www.rfc-editor.org/rfc/rfc8785

GENERATIVE AI USAGE

Generative AI tools were used as writing and engineering assistants during development and drafting. The authors remained responsible for all claims, experiments, code review, evidence interpretation, and final manuscript content. No confidential peer-review material was shared with AI tools.